

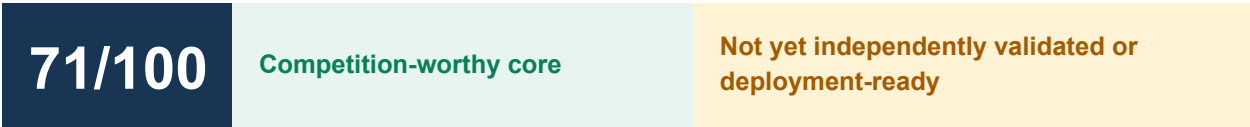
INTERNAL DECISION BRIEF | EVIDENCE-LED | JUDGE-SAFE

ContextShield

judging-readiness audit

Where the SIH 2026 solution stands today - what is proved, what is controlled, and what will move the score.

Assessment date	3 September 2026
Problem statement	SIH26171 - On-device Visual Perception for Light-weight Browser Agents
Evidence base	Repository inspection, fresh tests, primary research, and first-party guidance



Bottom line

The architecture is the strongest part. The largest score risk is the gap between controlled 100% regression results and independent, pixel-level, real-world evidence.

Recommended positioning

"A working privacy-boundary architecture with controlled proof and a clear path to independent validation."

Executive answer

ContextShield is a credible and unusually complete hackathon prototype, but it is not yet a defensible “works everywhere” or finale-ready product. A conservative score against the supplied ten-part internal rubric is **71/100**, with a plausible range of **68 - 76** depending on demo reliability and the judges’ emphasis. The core architecture is the strongest part: local screenshot/DOM collection, local privacy processing, strict sanitized schemas, a single outbound gateway, local secret handles, typed server actions, confirmation gates, and post-action verification are all implemented rather than merely diagrammed.

The main weakness is evidence quality. The repository reports 100% controlled recall, PII precision/recall, and redaction performance, but these are not independent real-world measures. The 60-screen perception suite is generated HTML whose expected labels are recovered primarily through DOM extraction. The 295-case PII corpus is programmatically created to match the deterministic detectors. The 100 pixel cases verify that supplied boxes are blacked out; they do not measure whether the system finds the right boxes on realistic screenshots. The repository itself correctly admits that independently reviewed YOLOX accuracy, broad task completion, clean-machine reproduction, and Firefox runtime acceptance are unavailable.

The product should be pitched as **“a working privacy-boundary architecture with controlled proof and a clear path to independent validation,”** not as a universal browser agent with proven 100% accuracy. This wording increases credibility.

Scope and method

The audit combines four evidence classes:

1. The judging rubric supplied by the team: ten criteria worth ten marks each.
2. The SIH26171 statement and its five technical weights, using a current archive because the official SIH page was not retrievable during this audit.
3. Direct repository inspection and fresh local verification on 3 September 2026.
4. Current primary research and first-party product documentation on browser-agent privacy, local inference, and comparable systems.

The assessment does not treat repository documentation as proof by itself. Claims are marked **proved**, **controlled**, **partial**, or **not evidenced**.

Headline scorecard

Criterion	Score	Current judgment
Problem Understanding	9/10	Excellent articulation of the privacy failure mode, trust boundary, users, and fail-closed behavior.
Innovation & Originality	7/10	Strong integrated implementation, but local redaction, placeholders, and guarded execution now have close published/open-source prior art.
Relevance to Problem Statement	8/10	Directly addresses almost every requested component; the local vision model is presently face/privacy-oriented rather than a general visual UI-understanding model.
Technical Approach & Architecture	8/10	Typed, layered, fail-closed architecture is strong; visual grounding, permission scope, and deployment boundaries require hardening.
Feasibility (Time & Resources)	8/10	One-command local setup and packaged privacy models are practical; Qwen, browser footprint, and clean-machine setup are meaningful constraints.
Prototype / Proof of Concept	8/10	Chrome build, unit/backend/browser tests and controlled end-to-end flows exist; broad unknown-site reliability remains unproved.
Scalability	5/10	No multi-user service, remote deployment, store distribution, extension update path, or independent Firefox runtime proof.
Impact & Usefulness	8/10	The privacy need is real and current; no user study, pilot, or quantified beneficiary outcome is supplied.
Business / Sustainability Model	3/10	No defined customer, pricing, procurement, support, unit economics, or maintenance plan was found.
Presentation & Communication	7/10	UI and documentation are strong; overbroad "all websites" language and controlled 100% numbers create avoidable trust risk.
Total	71/100	Strong prototype; not yet evidence-complete or deployment-ready.

This is an analytical estimate, not an official SIH score.

What the code genuinely proves

Proved in the fresh audit

- `./scripts/check.sh` completed successfully on 3 September 2026.
- TypeScript lint and type checking passed.
- Four shared-schema tests and forty extension unit tests passed.
- Thirty Python backend tests passed.
- Chrome and Firefox MV3 production artifacts built.
- Five default Chrome browser tests passed; eight real-stack/live tests were skipped because the Qwen, backend, and demo services were offline.
- The Git repository was clean, with 157 tracked files and the local `main` matching `origin/main` at commit `6c71f4f`.

Architecturally strong and directly inspectable

- `extension/lib/network/privacyGateway.ts` is a narrow outbound boundary. It parses a strict `SanitizedContext`, blocks suspicious keys and obvious sensitive strings, checks known local secrets, limits payload size, omits credentials and referrers, and only permits HTTPS or loopback HTTP.
- `shared/schemas/index.ts` uses strict Zod unions for sanitized context, action types, and verification results. The remote model cannot return selectors, JavaScript, arbitrary URLs, or untyped commands.
- `extension/lib/actions/broker.ts` verifies origin, snapshot, fingerprint, element existence, visibility, enabled state, requested options, confirmation, and post-action page changes.
- `extension/lib/privacy/visualPrivacyScanner.ts` captures a viewport locally, runs packaged YOLOX face detection and PP-OCR, turns OCR/vision output into non-actionable records, and only produces verified redacted crops.
- `extension/workers/pii.worker.ts` disables remote model loading and packages the Rampart contextual model locally.
- `server/app/model/qwen.py` constrains Qwen output with a JSON schema, validates grounding, treats page text as untrusted, and falls back to conservative deterministic actions rather than executing an ungrounded model response.
- The local secret vault and `LOCAL_*` handles preserve the ability to fill private values without revealing them to the planner.

These are meaningful technical differentiators. They directly address a broader industry risk: a 2025 study of eight browser agents found thirty privacy concerns and at least one concern in every system evaluated [Privacy Practices of Browser Agents](#).

Where the evidence is weaker than the headline

1. Visual understanding is not yet validated as visual understanding

SIH26171 asks for a client-side vision model that evaluates screen state; visual-context accuracy carries 25% of the stated technical evaluation [SIH26171 archive](#). ContextShield does run local pixel models, but the current YOLOX artifact has only one label, FACE. PP-OCR extracts visible text. Most actionable UI semantics still come from DOM/ARIA extraction, and pixel-derived records are deliberately disabled unless fused to a DOM record.

The claimed 167/167 relevant-interactable recall on 60 screens is useful regression evidence, but the test generates simple HTML and searches for expected labels in the semantic element list. It does not compare pixel-predicted boxes/classes against human annotations, and 47 of the 60 layouts are mechanically generated variations of the same three controls. The machine-readable report also leaves `screen_context_accuracy` as null.

Judge-safe status - partial. Real local pixel processing is proved; general visual screen understanding is not.

2. PII accuracy is controlled, not general

The 295 cases are created in code: fixed counts of emails, phones, PAN-like values, IFSC, UPI, OTP, password, account, person, address, medical text, five cards, and 100 benign product strings. This is a solid unit/regression suite. It is not an independent test set and does not measure Rampart, OCR errors, multilingual content, layout variation, partial values, screenshots, or adversarial near-misses.

The gap is material. The 2026 WebPII benchmark contains 44,865 annotated web UI images and reports that a visual model materially outperformed OCR/text baselines; the paper's model achieved 0.753 mAP@50 versus 0.357 for its stronger baseline [WebPII](#). A separate 2026 multi-source text benchmark reports very low span-level F1 for common baselines, illustrating how much harder real cross-domain PII detection is than template-generated cases [PIIBench](#).

Judge-safe status - controlled proof only. Do not say “100% PII accuracy” without immediately saying “on our synthetic regression corpus.”

3. Redaction correctness is proved only after detection supplies a box

The raster benchmark creates a known rectangular box, blackens it, checks every sensitive pixel is black, and confirms non-sensitive pixels are unchanged. That proves irreversible masking code. It does not measure end-to-end detection-plus-redaction precision, missed pixels around text/face boundaries, OCR localization errors, or utility retained after masking.

The actual system conservatively drops private-document regions when uncertain, which is privacy-safe but may reduce agent utility. The current face detector has no independently labelled accuracy result and no dedicated private-document detector.

Judge-safe status - masking implementation passed; real redaction precision unavailable.

4. Resource evidence needs a better denominator

The recorded aggregate Chromium test-profile peak is about 1.356 GB RSS, average CPU is 77.5% where 100% equals one logical core, peak GPU-process RSS is about 157 MB, and the unpacked compact Chrome release is about 104 MB. These figures include renderer, network, browser, extension, and GPU processes, so they do not isolate ContextShield's incremental cost. They are still too large to support an unqualified “lightweight” claim.

The implementation also runs face detection and OCR over the full downscaled viewport on a visual pass. This differs from documentation that describes targeted OCR as the normal cascade. The appropriate evidence is an A/B measurement: browser idle baseline versus extension active; cold start versus warm step; WebGPU versus WASM; DOM-only versus visual-triggered page.

ONNX Runtime Web makes browser-side GPU/WASM inference technically feasible, but its own documentation frames WebGPU/WASM as selectable execution providers rather than evidence that a particular application is lightweight [ONNX Runtime Web](#).

Judge-safe status - measured but not isolated; optimization/evaluation gap.

5. End-to-end autonomy is demonstrated narrowly

The stored live report shows three controlled tasks completed. Median task latency was 3.668 seconds, while the five-step Qwen checkout took about 14.1 seconds and its main Qwen round trip took about 10.9 seconds. Public-site regressions exist for DemoQA, WebDriver University, and the-internet.herokuapp.com, but the audit's default gate skips them unless the full local stack is running.

This is credible PoC evidence, not broad autonomy. The recent interaction history also showed multiple stagnation, clarification, perception, and verification failures before targeted fixes were added. Those fixes are good engineering, but they demonstrate that unknown-site generalization is still the central risk.

This difficulty is normal: OpenAI's first-party CUA report gave 58.1% on WebArena, well below human performance, even for a frontier screenshot-based agent [Computer-Using Agent](#). ContextShield should therefore report held-out task success honestly rather than promise universal compatibility.

Judge-safe status - working controlled prototype; unknown-site pass@1 unavailable.

Rubric-by-rubric assessment

1. Problem Understanding - 9/10

The pain point is explicit: cloud browser agents may receive screenshots, page content, and form data. First-party product documentation confirms this is real rather than hypothetical: ChatGPT agent uses browser screenshots and retains them with the conversation until deletion, while Claude Computer Use processes and collects screenshots used for interaction [ChatGPT agent](#), [Anthropic Computer Use privacy](#). ContextShield correctly defines the local client as the privacy authority and the planner as untrusted.

Lost mark: the user/product scope should be stated more narrowly: privacy-sensitive browser workflows on compatible ordinary web pages, not “everything everywhere.”

2. Innovation & Originality - 7/10

The project’s novelty is the combination of strict client authority, semantic minimization, real pixel masking, opaque local handles, typed action grounding, fail-closed behavior, and local verification in a Chrome/Firefox extension. However, it is not first-of-kind. PrivWeb uses a localized model to anonymize interface information and adds user controls; SafeScreen uses local DOM-assisted screenshot redaction, placeholders, an action guard, and local placeholder substitution [PrivWeb](#), [SafeScreen](#). Related mobile-agent research also uses type-preserving placeholders and a secure local interaction proxy [Available but Invisible](#).

Best originality claim: “ContextShield turns privacy into a client-enforced execution invariant, not a promise made by the remote model.” Avoid “first ever.”

3. Relevance - 8/10

The solution maps directly to the requested extension, client-side inference, dynamic PII redaction, sanitized server context, structured commands, and end-to-end task loop. The one material mismatch is that the vision detector is face-only and the actionable screen map is mostly DOM/ARIA/OCR-based; it does not yet prove a ViT-equivalent general visual UI model “reads” the screen and makes decisions.

4. Technical Approach & Architecture - 8/10

The strict schemas, single gateway, worker isolation, local vault, state fingerprints, confirmation gates, model grounding, and fail-closed errors are excellent hackathon engineering. The most concerning architecture choices are broad `<all_urls>` host permission, loopback-only CSP in the shipped build despite discussion of a remote centralized planner, and weak support for inaccessible frames, closed shadow roots, browser-owned PDF surfaces, and custom canvas-only controls.

Chrome explicitly recommends `activeTab` as a temporary-access alternative to broad persistent host access because compromise of an all-sites extension has a larger blast radius [Chrome activeTab](#). ContextShield already requests `activeTab`; it should justify or reduce the simultaneous all-sites grant.

5. Feasibility - 8/10

The repository provides a one-command macOS start flow, packaged local privacy models, reproducible npm/Python setup, a stable extension release, and mock mode. The 3 GB Qwen download, 100 MB unpacked extension, significant aggregate browser memory, platform-specific launchctl behavior, and requirement for current llama.cpp make classroom deployment less trivial than the UI suggests. Clean second-machine reproduction is still missing.

6. Prototype / Proof of Concept - 8/10

This is a real implementation, not slides: it builds both browser targets, passes unit/backend/browser tests, runs local ONNX models, emits sanitized requests, executes typed actions, and includes controlled demos. The deduction reflects narrow held-out coverage and the fact that the full real-stack suite was not running during the fresh audit.

7. Scalability - 5/10

The design can scale conceptually through a remote HTTPS planner, but the shipped CSP only connects to loopback, the backend uses local SQLite and lacks production auth/multitenancy/observability, the model server is single-machine llama.cpp, Firefox is build-only, and there is no store release or policy/managed-extension deployment. Browser UI restrictions also prevent literal all-site coverage.

8. Impact & Usefulness - 8/10

The value is strong for government portals, banking, healthcare, HR, customer support, and regulated enterprise workflows where browser automation meets sensitive data. The 2025 Indian DPDP Rules create a timely compliance context and phased obligations [MeitY DPDP Rules 2025](#). Research showing thirty privacy concerns across eight browser agents strengthens the impact case. No pilot, user study, time saved, or risk-reduction measure currently validates the benefit.

9. Business / Sustainability - 3/10

No business model was found. A credible plan would name a beachhead customer, buyer, deployment model, pricing unit, operating-cost assumptions, support responsibility, model/update policy, and adoption path. A strong initial model is B2B/B2G: free evaluator extension plus paid managed policy packs, on-prem/private-cloud planner, audit reports, and enterprise support. Do not lead with a consumer subscription before proving accuracy and trust.

10. Presentation & Communication - 7/10

The popup, safe activity log, privacy counts, server-view panel, README, setup guide, threat model, and acceptance report are strong. Three communication issues lower confidence: controlled results appear close to headline claims; some docs say targeted OCR while implementation performs full-viewport OCR; and the acceptance document's ZIP hash/size no longer matches the current release (f424... documented versus current 7c56...). A judge can interpret such inconsistencies as weak experimental discipline even when the software works.

Competitive position

Alternative	Typical strength	Privacy limitation	ContextShield position
Cloud screenshot agents	Strong general visual reasoning and mature task loops	Screenshots/page state are processed off-device; sensitive workflows require safeguards/takeover	Stronger architectural minimization, weaker general task accuracy
PrivWeb	Local privacy categorization, notifications, studied UX	Primarily an add-on/privacy-control layer rather than the same strict action architecture	Similar privacy concept; ContextShield's typed gateway/action invariant is the differentiator
SafeScreen	Redacted screenshots, placeholders, action guard	Smart detection mode can place raw input inside a separately trusted remote boundary	ContextShield currently keeps detection models packaged in the extension and fails closed
DOM-first local browser agents	Fast, compact, grounded control	Miss pixels, canvas, images, faces, and visual-only PII	ContextShield adds local pixel/OCR privacy analysis, but has not yet proved visual UI accuracy

Priority plan before judging

P0 - required to defend the core claim

1. **Replace headline “100%” language.** Say “100% on controlled regression corpus; independent accuracy pending.”
2. **Build an independent visual benchmark.** Start with a fixed, license-reviewed WebPIL subset plus at least 100 team-labelled screenshots across English/Hindi, forms, images, canvas, PDF, faces, and near-miss negatives. Report box mAP/loU, per-class precision/recall, leakage pixels, and utility retained.
3. **Prove visual screen understanding.** Add/benchmark a lightweight UI-region model that detects at least text regions, input fields, buttons, and images from pixels. Show a canvas/image-only control recognized locally with the server disconnected.
4. **Run held-out task evaluation.** Freeze twenty tasks across at least five public/independently authored sites. Report pass@1, completion time, failure categories, confirmations, and privacy leakage. Do not patch selectors after seeing the held-out set.
5. **Isolate resource cost.** Measure baseline Chrome versus Chrome+ContextShield, cold/warm, WebGPU/WASM, and DOM-only/vision-triggered paths. Report incremental RSS/CPU/GPU and energy where possible.
6. **Perform a real Firefox run.** Record installation, model initialization, privacy proof, action loop, and WASM latency on a second machine.

P1 - required to move from prototype to product story

7. Reduce or justify `<all_urls>`; evaluate optional host permissions and per-origin grants.
8. Reconcile documentation, release hash/size, benchmark wording, and implementation behavior.
9. Add a production deployment profile: HTTPS endpoint, authentication, tenant isolation, rate limits, signed policy/model updates, trace retention, and no raw-data observability.
10. Create a one-page business case: first buyer, pilot, pricing, cost per task, support plan, and measurable outcome.

P2 - useful if time remains

11. Add multilingual OCR/PII evaluation and Indian government-form fixtures.
12. Add prompt-injection and privacy-exfiltration red-team cases modeled on current browser-agent research.
13. Add signed release artifacts, SBOM/provenance, GitHub release tag, branch protection, and store-submission checklists.

Recommended judging demo

Use a six-minute proof sequence:

1. **Problem (30 seconds):** a normal cloud agent sees raw screenshots and forms.
2. **Privacy proof (90 seconds):** a page with name, email, phone, password, face, and canvas text; show local model status, then inspect the exact sanitized request.
3. **Utility proof (90 seconds):** execute a multi-step task using a local secret handle and a sanitized Qwen decision.

4. **Safety proof (45 seconds):** trigger stale state or an unsafe submit and show the local confirmation/fail-closed gate.
5. **Measured results (60 seconds):** show independent accuracy and incremental resource/latency numbers with dataset sizes.
6. **Close (45 seconds):** explain deployment path and one target customer.

The most important live sentence is: “**The server is useful but never trusted with identity; the extension decides what can cross the boundary and what action can execute.**”

Questions judges are likely to ask

- Is your screen-context score based on pixels or DOM labels?
- What happens to PII inside images, canvas, PDFs, iframes, and multiple languages?
- How do you know redaction did not leave recoverable pixels or crop edges?
- What is your false-negative rate on a dataset your detector did not generate?
- What is the incremental extension RAM/CPU, not total Chrome usage?
- Why is `<all_urls>` required if `activeTab` is already present?
- Can the planner be compromised without exposing secrets or executing arbitrary commands?
- Does Firefox actually run, or does it only compile?
- Which parts work when Qwen is offline?
- Who pays, what do they buy, and how is policy/model maintenance funded?

Final verdict

ContextShield has a **competition-worthy core** and one of the better safety architectures a hackathon team could show. Its competitive weakness is not lack of implementation; it is the distance between controlled regression evidence and the broader claims judges will naturally infer. If the team spends the next iteration on independent benchmarks, real visual UI understanding, resource isolation, held-out task success, Firefox proof, and a concrete adoption model, the likely rubric outcome can move from the low 70s into the low-to-mid 80s without rebuilding the architecture.

Claim-to-source ledger

Claim family	Source	Publisher / date	Access note
SIH26171 requirements and technical weights	SIH26171 archive	Community archive of SIH content, accessed 3 Sep 2026	Official SIH page was unavailable to the research tool; wording should be rechecked against the official page before submission.
Browser-agent privacy risks and 30 concerns	Privacy Practices of Browser Agents	Ukani et al., 8 Dec 2025	Research preprint; methodology and counts visible.
Cloud agents use/process screenshots	ChatGPT agent ; Anthropic privacy	OpenAI, accessed 3 Sep 2026; Anthropic, 16 Mar 2026	First-party product/privacy documentation.
General browser-agent task difficulty	Computer-Using Agent	OpenAI, 23 Jan 2025	First-party benchmark report.
Comparable local privacy systems	PrivWeb ; SafeScreen	Zhang et al., 15 Sep 2025; open-source repository accessed 3 Sep 2026	Demonstrates close prior art; implementation/trust boundaries differ.
Visual PII benchmark scale/baseline	WebPII	Zhao, 18 Mar 2026	Research paper and released benchmark/model claim.
Text PII benchmark difficulty	PIIBench	Jha, 17 Apr 2026	Research preprint; useful benchmark candidate.
Browser-side inference feasibility	ONNX Runtime Web	Microsoft ONNX Runtime documentation, accessed 3 Sep 2026	First-party technical documentation.
Extension permission trade-off	Chrome activeTab	Chrome for Developers, accessed 3 Sep 2026	First-party extension security guidance.
Indian privacy-policy context	DPDP Rules 2025	MeitY / Gazette of India, 13 Nov 2025	Official government notification; business relevance, not a legal compliance opinion.

Repository evidence reviewed

README.md; START-HERE.md; docs/SIH-ACCEPTANCE-v1.2.0.md; docs/target-status.md; docs/architecture.md; docs/privacy-model.md; docs/threat-model.md; benchmarks/run.ts; benchmarks/results/latest.json; benchmarks/results/live-product.json; extension/tests/compatibility.spec.ts; extension/tests/live.spec.ts; extension/lib/network/privacyGateway.ts; extension/lib/privacy/pipeline.ts; extension/lib/privacy/visualPrivacyScanner.ts; extension/workers/vision.worker.ts; extension/workers/ocr.worker.ts; extension/workers/pii.worker.ts; extension/lib/actions/broker.ts; extension/lib/agent/localController.ts; shared/schemas/index.ts; server/app/model/qwen.py; .github/workflows/ci.yml; and release artifacts.

Limitations

- No judge interviews or internal scoring guidance beyond the supplied rubric were available.

- The official SIH page could not be directly retrieved during research; a current archive reproducing the full statement was used.
- The full Qwen/live suite was not rerun because all three local services were offline; stored 31 August results and source tests were inspected, while the default automated gate was rerun fresh.
- No independent dataset annotations, user study, second machine, Firefox runtime session, cloud deployment, or browser-store submission were available.

Internal analytical estimate. This is not an official SIH score, legal opinion, or independent certification.